

CTR Native Mod Engine

Guía de Modding / Modding Guide

Versión 5.0

Sistema de mods Lua para Crash Team Racing PC Port
Lua mod system for Crash Team Racing PC Port

PS1 OT: 512×216 px

CTR Native Community

All coordinate values use PS1 OT coordinate system (512x216 pixels)

Contenido / Table of Contents

- 1 Introducción / Introduction
- 2 Estructura de un Mod / Mod Structure
- 3 Sistema de Hooks / Hook System
- 4 API de Lectura / Read API
- 5 API de Escritura / Write API
- 6 API de Dibujo / Drawing API
- 7 API de Utilidades / Utility API
- 8 Override de Archivos / File Override System
- 9 Ejemplo Completo / Complete Example
- 10 Notas Técnicas / Technical Notes

Nota importante / Important note: Todas las coordenadas usan el sistema PS1 OT: **512×216 píxeles**. Centro: X=0x100 (256), Y=0x6C (108).

All coordinates use the PS1 OT system: **512×216 pixels**. Center: X=0x100 (256), Y=0x6C (108).

1. Introducción / Introduction

ES

El motor de mods de CTR Native permite modificar el comportamiento del juego mediante scripts Lua que se ejecutan en tiempo real. Los mods se colocan en la carpeta **mods/** junto al ejecutable y se cargan automáticamente al iniciar el juego. No es necesario recompilar el proyecto para agregar o modificar mods — el binario compilado puede ejecutar cualquier mod Lua que se coloque en la carpeta correcta.

El sistema soporta tres tipos de modificaciones:

- **Lectura de estado del juego:** Observar variables como reservas, velocidad, modo de juego
- **Escritura en memoria:** Modificar el comportamiento cambiando valores cada frame
- **Dibujo en pantalla:** Renderizar elementos visuales custom en el HUD

EN

The CTR Native mod engine allows modifying game behavior through Lua scripts that execute in real-time. Mods are placed in the **mods/** folder next to the executable and are loaded automatically at startup. No recompilation is needed to add or modify mods — the compiled binary can run any Lua mod placed in the correct folder.

The system supports three types of modifications:

- **Read game state:** Observe variables like reserves, speed, game mode
- **Write to memory:** Modify behavior by changing values every frame
- **Draw on screen:** Render custom visual elements on the HUD

2. Estructura de un Mod / Mod Structure

ES

Cada mod es una carpeta dentro de **mods/** que contiene al menos un archivo **main.lua**. Opcionalmente puede tener una carpeta **files/** para assets del mod (imágenes, configs, overrides de BIGFILE).

```
mods/  
  mi_mod/  
    main.lua          -- Script principal (obligatorio)  
    files/            -- Assets del mod (opcional)  
    config.txt  
    BIGFILE/          -- Override de assets del juego (opcional)  
      lang/en.lng     -- Reemplaza el archivo de idioma  
      levels/tracks/  -- Reemplaza pistas, etc.
```

EN

Each mod is a folder inside **mods/** containing at least a **main.lua** file. Optionally it can have a **files/** folder for mod assets (images, configs, BIGFILE overrides).

```
mods/  
  my_mod/  
    main.lua          -- Main script (required)  
    files/            -- Mod assets (optional)  
    config.txt  
    BIGFILE/          -- Game asset overrides (optional)  
      lang/en.lng     -- Replaces language file  
      levels/tracks/  -- Replaces tracks, etc.
```

3. Sistema de Hooks / Hook System

ES

Los mods registran callbacks que se ejecutan en puntos específicos del game loop:

Hook	Cuándo se ejecuta	Uso típico
onInit	Al cargar los mods	Inicialización, leer configs
onUpdate	Cada frame, durante la lógica del juego	Modificar estado del juego
onRender	Cada frame, durante el renderizado	Dibujar en el HUD
onInput	Al procesar input del gamepad	Keybinds custom
onTitleInit	Al iniciar la pantalla de título	Reset de estado

EN

Mods register callbacks that execute at specific points in the game loop:

Hook	Cuándo se ejecuta	Uso típico
onInit	When mods are loaded	Initialization, read configs
onUpdate	Every frame, during game logic	Modify game state
onRender	Every frame, during rendering	Draw on HUD
onInput	When gamepad input is processed	Custom keybinds
onTitleInit	When title screen initializes	State reset

Example

```
mod.hook("onInit", function()
  mod.log("My mod loaded!")
end)

mod.hook("onUpdate", function()
  -- Runs every frame during game logic
end)

mod.hook("onRender", function()
  -- Runs every frame during rendering
end)
```

4. API de Lectura / Read API

ES

Estas funciones permiten observar el estado del juego sin modificarlo:

Función	Descripción
<code>mod.getDriver(index)</code>	Retorna tabla con datos del jugador (0-7). Campos: valid, reserves, fireSpeedCap, turbo_MeterRoomLeft, turbo_outsideTimer, numTurbos, const_SacredFireSpeed, const_SingleTurboSpeed, speedApprox, actionsFlagSet
<code>mod.getNumPlayers()</code>	Número de jugadores en la partida
<code>mod.getGameMode()</code>	Modo de juego actual (gameMode1)
<code>mod.getGameTracker()</code>	Puntero (lightuserdata) al GameTracker. Permite acceso total al estado del juego combinado con readS16/readU8/readS32
<code>mod.readS16(ptr, offset)</code>	Lee un valor signed 16-bit de memoria
<code>mod.readU8(ptr, offset)</code>	Lee un valor unsigned 8-bit de memoria
<code>mod.readS32(ptr, offset)</code>	Lee un valor signed 32-bit de memoria

Offsets conocidos del GameTracker

Offset	Campo
0x000	gameMode1 (s32)
0x343	numPlyrCurrGame (u8)
0x24EC	drivers[] (array de 8 punteros)

EN

These functions allow observing game state without modifying it:

Función	Descripción
<code>mod.getDriver(index)</code>	Returns table with player data (0-7). Fields: valid, reserves, fireSpeedCap, turbo_MeterRoomLeft, turbo_outsideTimer, numTurbos, const_SacredFireSpeed, const_SingleTurboSpeed, speedApprox, actionsFlagSet
<code>mod.getNumPlayers()</code>	Number of players in current game
<code>mod.getGameMode()</code>	Current game mode (gameMode1)
<code>mod.getGameTracker()</code>	Pointer (lightuserdata) to GameTracker. Allows full game state access combined with readS16/readU8/readS32
<code>mod.readS16(ptr, offset)</code>	Read a signed 16-bit value from memory
<code>mod.readU8(ptr, offset)</code>	Read an unsigned 8-bit value from memory
<code>mod.readS32(ptr, offset)</code>	Read a signed 32-bit value from memory

Known GameTracker offsets

Offset	Campo
0x000	gameMode1 (s32)
0x343	numPlyrCurrGame (u8)
0x24EC	drivers[] (array of 8 pointers)

5. API de Escritura / Write API

ES

Estas funciones permiten **MODIFICAR** el comportamiento del juego escribiendo en memoria. Usar con cuidado — un mod mal escrito puede causar crashes.

Función	Descripción
<code>mod.setDriverField(index, name, value)</code>	Escribe un campo del driver de forma segura por nombre. Campos escribibles: "reserves", "fireSpeedCap", "turbo_MeterRoomLeft", "turbo_outsideTimer", "numTurbos", "kartState", "actionsFlagSet", "speedApprox". Retorna true/false
<code>mod.writeS16(ptr, offset, value)</code>	Escribe un valor signed 16-bit en cualquier dirección de memoria
<code>mod.writeU8(ptr, offset, value)</code>	Escribe un valor unsigned 8-bit en cualquier dirección
<code>mod.writeS32(ptr, offset, value)</code>	Escribe un valor signed 32-bit en cualquier dirección

Modelo de modificación: El motor funciona con un modelo de "observar y corregir": el juego ejecuta su lógica C normalmente, y después de cada frame tu mod Lua puede leer los resultados y modificar los datos antes del siguiente frame.

EN

These functions allow **MODIFYING** game behavior by writing to memory. Use with care — a poorly written mod can cause crashes.

Función	Descripción
<code>mod.setDriverField(index, name, value)</code>	Safely writes a driver field by name. Writable fields: "reserves", "fireSpeedCap", "turbo_MeterRoomLeft", "turbo_outsideTimer", "numTurbos", "kartState", "actionsFlagSet", "speedApprox". Returns true/false
<code>mod.writeS16(ptr, offset, value)</code>	Write a signed 16-bit value to any memory address
<code>mod.writeU8(ptr, offset, value)</code>	Write an unsigned 8-bit value to any address
<code>mod.writeS32(ptr, offset, value)</code>	Write a signed 32-bit value to any address

Modification model: The engine works on an "observe and correct" model: the game runs its C logic normally, and after each frame your Lua mod can read the results and modify the data before the next frame.

Examples


```
-- Infinite reserves
mod.hook("onUpdate", function()
    local d = mod.getDriver(0)
    if d.valid and d.reserves > 0 then
        mod.setDriverField(0, "reserves", 9600)
    end
end)

-- Increase max speed
mod.hook("onUpdate", function()
    mod.setDriverField(0, "fireSpeedCap", 0x4000)
end)

-- Full GameTracker access
mod.hook("onUpdate", function()
    local gGT = mod.getGameTracker()
    if gGT then
        mod.writeU8(gGT, 0x343, 4) -- force 4 players
    end
end)
```

6. API de Dibujo / Drawing API

ES

Estas funciones permiten dibujar elementos visuales en el HUD del juego. Las coordenadas usan el sistema PS1 OT: **512×216 píxeles**. Centro: X=0x100 (256), Y=0x6C (108).

Función	Descripción
<code>mod.drawRect(x, y, w, h, r, g, b, [a])</code>	Dibuja un rectángulo coloreado. a=255 (opaco) por defecto
<code>mod.drawText(text, x, y, [font], [justify])</code>	Dibuja texto. font: 1=grande, 2=pequeño. justify: 0=izquierda, 1=centro, 2=derecha

EN

These functions allow drawing visual elements on the game HUD. Coordinates use PS1 OT system: **512×216 pixels**. Center: X=0x100 (256), Y=0x6C (108).

Función	Descripción
<code>mod.drawRect(x, y, w, h, r, g, b, [a])</code>	Draw a colored rectangle. a=255 (opaque) by default
<code>mod.drawText(text, x, y, [font], [justify])</code>	Draw text. font: 1=big, 2=small. justify: 0=left, 1=center, 2=right

Example

```
mod.hook("onRender", function()
  -- Draw a green reserve bar
  mod.drawRect(10, 104, 60, 6, 0, 255, 0)
  -- Draw label
  mod.drawText("RESERVE", 74, 103, 2, 0)
end)
```

7. API de Utilidades / Utility API

Función / Function	Descripción / Description
<code>mod.log(msg)</code>	Imprimir mensaje en consola / Print message to console
<code>mod.getModPath()</code>	Ruta del mod actual / Current mod path
<code>mod.readFile(path)</code>	Leer archivo de files/ / Read file from files/
<code>mod.writeFile(path, data)</code>	Escribir archivo en files/ / Write file to files/
<code>mod.hook(name, func)</code>	Registrar callback / Register callback

8. Override de Archivos / File Override System

ES

Los mods pueden reemplazar archivos del juego colocándolos en su carpeta **files/**. El sistema busca archivos en este orden de prioridad:

1. `mods/mi_mod/files/` — Override desde el mod (máxima prioridad)
2. `assets/BIGFILE/` — Carpeta desempaquetada global (HYBRID/UNPACKED)
3. `assets/` — Assets base del juego

Por ejemplo, para reemplazar el archivo de idioma inglés:

Crear: `mods/mi_mod/files/BIGFILE/lang/en.lng`

El juego cargará este archivo en vez del original

Para reemplazar una pista:

Crear: `mods/mi_mod/files/BIGFILE/levels/tracks/coco/1P/data.lev`

EN

Mods can replace game files by placing them in their **files/** folder. The system searches files in this priority order:

1. `mods/my_mod/files/` — Mod override (highest priority)
2. `assets/BIGFILE/` — Global unpacked folder (HYBRID/UNPACKED)
3. `assets/` — Base game assets

For example, to replace the English language file:

Create: `mods/my_mod/files/BIGFILE/lang/en.lng`

The game will load this file instead of the original

To replace a track:

Create: `mods/my_mod/files/BIGFILE/levels/tracks/coco/1P/data.lev`

9. Ejemplo Completo / Complete Example — Reserve Bar

ES

Este mod dibuja una barra visual en el HUD que indica las reservas actuales del jugador, con colores según el nivel de fuego:

```
-- Configuración
local CONFIG = {
    barX = 0x0A, barY = 0x68,
    barWidth = 0x3C, barHeight = 0x06,
    colorNormal = { 0x00, 0xFF, 0x00 },      -- Verde
    colorSacredFire = { 0xFF, 0xFF, 0x00 },  -- Amarillo
    colorUltraSacred = { 0xFF, 0x40, 0x00 }, -- Naranja
    maxReserveDisplay = 4800,
}

local function getReserveColor(driver)
    if driver.fireSpeedCap > driver.const_SacredFireSpeed then
        return CONFIG.colorUltraSacred
    elseif driver.fireSpeedCap > driver.const_SingleTurboSpeed then
        return CONFIG.colorSacredFire
    end
    return CONFIG.colorNormal
end

mod.hook("onRender", function()
    local driver = mod.getDriver(0)
    if not driver.valid or driver.reserves <= 0 then return end

    local ratio = driver.reserves / CONFIG.maxReserveDisplay
    local fillW = math.floor(ratio * (CONFIG.barWidth - 2))
    local color = getReserveColor(driver)

    -- Outline
    mod.drawRect(CONFIG.barX, CONFIG.barY,
        CONFIG.barWidth, CONFIG.barHeight, 0, 0, 0)
    -- Fill
    mod.drawRect(CONFIG.barX + 1, CONFIG.barY + 1,
        fillW, CONFIG.barHeight - 2,
        color[1], color[2], color[3])
end)
```

EN

This mod draws a visual bar on the HUD showing the player's current reserves, with colors based on fire level. Same code as above.

10. Notas Técnicas / Technical Notes

ES

- Coordenadas PS1 OT: **512×216 píxeles**. Centro: X=256, Y=108
- Los hooks se ejecutan en: MainMain.c (onUpdate), RenderFrame.c (onRender)
- El sistema bigfile soporta PACKED, UNPACKED y HYBRID
- BFDBG_PRINTF genera log detallado: `./ctr_native 2>&1 | grep "[BF-DBG]"`
- Los mods solo pueden modificar datos, no reemplazar funciones C
- El modelo es "observar y corregir" — el juego ejecuta su lógica y el mod la corrige después

EN

- PS1 OT coordinates: **512×216 pixels**. Center: X=256, Y=108
- Hooks execute at: MainMain.c (onUpdate), RenderFrame.c (onRender)
- Bigfile system supports PACKED, UNPACKED and HYBRID modes
- BFDBG_PRINTF generates detailed log: `./ctr_native 2>&1 | grep "[BF-DBG]"`
- Mods can only modify data, not replace C functions
- The model is "observe and correct" — the game runs its logic and the mod corrects it after

Generado por la comunidad CTR Native — Generated by the CTR Native Community
Versión 5.0 — PS1 OT Coordinate System (512×216)